

Speech commands classification with Transformers

Project raport

Jakub Rymuza

Rafał Sosnowski

August 19, 2026

Abstract

In this project the use of various deep learning models, especially Transformer models was used to solve the task of classifying speech commands. This work can be beneficial for many real-life task such as voice-operated machines. As a result of the project, a high accuracy of **89.06%** on the Kaggle competition test set was achieved. On the validation set the accuracy was even higher - **97.44%**. These values are high and satisfactory for many real-life use-cases.

The discrepancy between the result on the test set and the validation set might be because the test set was much bigger - it was around three times bigger than the training set and almost 30 times bigger than the validation set. The test set probably contained harder examples and examples that weren't similar to the ones found in the training set and the validation set.

Four models were tested: two transformer Emformer, Conformer, as well as InceptionTime model and our modification of it. Those models, except for Emformer all have achieved similarly high results. The best model was the modified InceptionTime.

Different approaches were considered to handle the special classes "unknown" and "silence". Out of those, the simplest single-network solution worked best.

Two data augmentation techniques from SpecAug suite were tested: TimeMasking and FrequencyMasking. Inclusion of those techniques improved the results of the model slightly. Moreover the influence of hyperparameter change was investigated. The ensembling technique was also used and slightly improved the results.

Contents

1	Introduction	3
2	Problem description	3
3	Solution	3
3.1	Models	3
3.1.1	InceptionTime	3
3.1.2	Conformer	4
3.1.3	Emformer	4
3.2	Preprocessing	5
3.3	Augmentations	5
3.4	Model training	6
3.5	Ensemble	6
4	Experiments	6
4.1	Architecture tests	7
4.2	Approaches tests	7
4.3	Augmentation tests	8
4.4	Hyperparameters tests	9
4.4.1	Epoch test	9
4.4.2	Batch Size test	10
4.4.3	Learning Rate test	11
4.5	Final model	11
4.6	Ensembled model	14
4.7	Reproducing experiments	16
5	Conclusions	16
6	Bibliography	17

1 Introduction

This project focuses on using various neural network architectures, including the Transformer architecture to classify speech commands effectively. Transformer architecture is deep learning architecture using the attention mechanism [1]. In today's digital world, audio processing is essential, especially in tasks like voice recognition. By utilizing Transformers, known for their ability to understand context and long-range dependencies, the intention was to improve the accuracy and efficiency of spoken word classification. The goal is to make speech recognition more reliable for everyday applications like voice-activated systems. Various techniques like data augmentation and model ensembling were used to achieve the optimal results. The influence of hyperparameter change on results was investigated. Different approaches to the classification process were used.

2 Problem description

In this project, various models for classifying speech commands were created. This means that there was a need to identify only some subset of words (commands) and not all the possible words. In the dataset [2] the goal is to identify 10 different commands. All other sounds need to be classified as either noise (silence) or an unknown word. In the training dataset we are provided with 31 labels: 10 command words, 20 different unknown words and the background noise. The samples were divided into 1 second `.wav` files, except background noises which are longer fragments of audio. The dataset is unbalanced, meaning the amount of data samples corresponding to each label is unequal, although it is roughly similar. The unknown words constitute over half of the dataset. The words were spoken by many different people and there were background noises present, to make the challenge more realistic.

3 Solution

This section describes the techniques and models used to solve the problem.

3.1 Models

Four neural network architectures were selected and tested: InceptionTime, InceptionTime with our modifications, Conformer and Emformer.

3.1.1 InceptionTime

InceptionTime is a deep learning model that uses the Inception-v4 architecture for time series classification. This model consists of an ensemble of deep convolutional network (CNN) models and is able to learn from large datasets in a short time.

InceptionTime uses convolutional techniques to analyze local and global patterns in time series data. Thanks to this, it is able to effectively detect and classify various patterns in data. For example, InceptionTime is able to learn from 1,500 time series

in one hour, as well as from 8 million time series in 13 hours. This makes it an extremely scalable and useful tool in many fields such as signal processing, financial forecasting, medical analysis and many others. [3]

The implementation of the architecture was based on a repository [4], although it was rewritten from the `Keras` library to the `PyTorch` library, in order to achieve compatibility with the rest of the project code, as well as to improve performance and modification flexibility.

Additionally, a modification of this network was created, introducing the following changes:

- changing the padding mode of first convolution layer - from padding the image with zeros (`padding_mode='zeros'`) to padding the image with pixels mirrored at the edges of the input data (`padding_mode='reflect'`),
- adding batch normalization ,
- initialization of the weight of convolutional layers using the Kaiming method.

In the document, the model with the above modifications will be called Modified Inception.

Additionally, later in the project, we added augmentations to the training process

3.1.2 Conformer

The Convolution-augmented Transformer for Speech Recognition (Conformer) architecture blends Transformer self-attention with convolutional layers, employing depthwise separable convolutions for local dependency capture [5]. It integrates feed-forward networks and multi-head self-attention for sequential data processing. Conformers feature depth-wise convolutions for temporal correlation modeling and dynamic receptive field adjustment, ensuring scalability for varying input lengths. Relative position bias enhances attention mechanisms. Effective across tasks, notably in speech and language understanding. This architecture is used for various speech recognition tasks.

The Conformer used in the project comes from the `PyTorch` library. The outputs of this implementation are of the same dimensions as the input (autoencoder architecture). Therefore, in order to use it for a classification task, the output was flattened and then put through two dense layers, followed by a softmax activation function.

3.1.3 Emformer

The Efficient Memory Transformer Based Acoustic Model for Low Latency Streaming Speech Recognition (Emformer) optimizes attention mechanisms with a hierarchical structure and dynamic routing for efficient processing in Transformer models [6]. It employs self-attention modules with varying receptive fields, capturing long-range dependencies effectively. Dynamic routing based on relevance enhances feature fusion. This architecture boosts performance across natural language processing tasks while

reducing computational complexity, making it versatile and efficient for diverse applications. This architecture is also widely used for various speech recognition tasks.

Analogically to the Conformer model, Enformer’s implementation also comes from the PyTorch library. Similarly, two dense layers were added at the end of the model.

3.2 Preprocessing

In order to prepare data for architectures, we:

- perform short-Time spectral analysis (STFT) of the audio signal - STFT is performed on a signal that is divided into short frames, and then a Fourier transform is applied to each frame. In this way, we obtain information about the frequency content of the signal depending on time.
- convert it to the decibel scale - The decibel scale is commonly used to represent the power or amplitude of a signal in a way that is more understandable to humans.
- convert the STFT to a mel-spectrogram - A mel-spectrogram is a representation of the sound spectrum in which the frequency axis is scaled in a way that better suits the human perception of sound. Thanks to this, the mel-spectrogram better reflects the way the human ear perceptually perceives differences in sound frequencies.

As a result of preprocessing, we obtain an array representing time and frequency respectively.

- The x-axis represents time, where each data point corresponds to subsequent time steps of the analyzed audio signal.
- The y-axis represents frequency, where each data point corresponds to a specific frequency in the audio spectrum being analyzed.

Some of the data loading and preprocessing code was based on a Kaggle notebook [7].

3.3 Augmentations

Two augmentation methods from the SpecAug suite [8] were used in the project:

- TimeMasking - This operation covers certain portions of the audio signal by applying a time-domain mask. The `time_mask_param` parameter specifies the maximum length of time that can be masked.
- FrequencyMasking - This operation covers certain frequencies in the spectrum of the audio signal. The `freq_mask_param` parameter specifies the maximum frequency width that can be masked.

3.4 Model training

During the project, various approaches to training models were tested:

- A single network that classified all 31 classes, then the output of the model was relabeled to match the 12 classes that were the goal (relabel the "unknown" classes).
- Relabel the "unknown" classes before training the model, so the model was trained on only 12 classes.
- Use a separate network to classify the "silence" class, then use a second network to classify all the other classes, except "silence".

3.5 Ensemble

In the project, the averaging model was chosen as the ensembling technique [9]. Model Averaging is one of the popular techniques used in Ensemble Learning, which is based on the principle of combining multiple models to achieve better predictive performance than single models operating individually. The main goal of this approach is to take advantage of the diversity in the results obtained by different models, which can lead to more stable and accurate predictions.

The main idea behind Model Averaging is that different models can make different errors on different sets of test data or in different regions of the input space. By averaging their predictions, the impact of random errors can be reduced, leading to more stable and reliable results.

In the case of the code described, the ensemble function performs modeling by calculating the arithmetic average of predictions from different models. This is a simple but effective way to put Model Averaging into practice.

4 Experiments

This section contains all the conducted experiments. The following assumptions were made for all experiments unless otherwise indicated:

- ADAM was chosen as the optimization algorithm.
- Linear scheduler was chosen as the learning rate scheduler.
- Cross-entropy was chosen as the loss function.
- The size of batch is 64.
- The learning process will last 30 epochs.
- The initial learning rate is 0.01 (which is decreased linearly by the scheduler during learning over 30 epochs to minimum value of 0.0001).

- The weight decay rate is $1e - 8$.

The training set contains almost 65000 cases, while the test set contains almost 160000 cases. The test set didn't contain labels (the results on the test set are only verifiable by submitting it to *Kaggle*, as this dataset is part of a competition). Therefore a validation dataset was constructed, which consisted of 10% of the files from the training dataset. A validation-training datasets split proposed by the authors was used, which ensured that a given speaker exists in only one of these datasets. Otherwise the validation results might have been artificially inflated.

There is a seed included in the program, allowing the reproduction of the results of all the experiments. All the below experiments are in a chronological order.

4.1 Architecture tests

Figure 1 below shows the accuracy results of each architecture. The best architecture is Modified Inception at around 96% accuracy. it is worth noting that the modifications did not significantly improve the operation of the original model, they improved it slightly, additionally Conformer as a transformer architecture worked very well, but slightly worse than architectures based only on convolution networks.

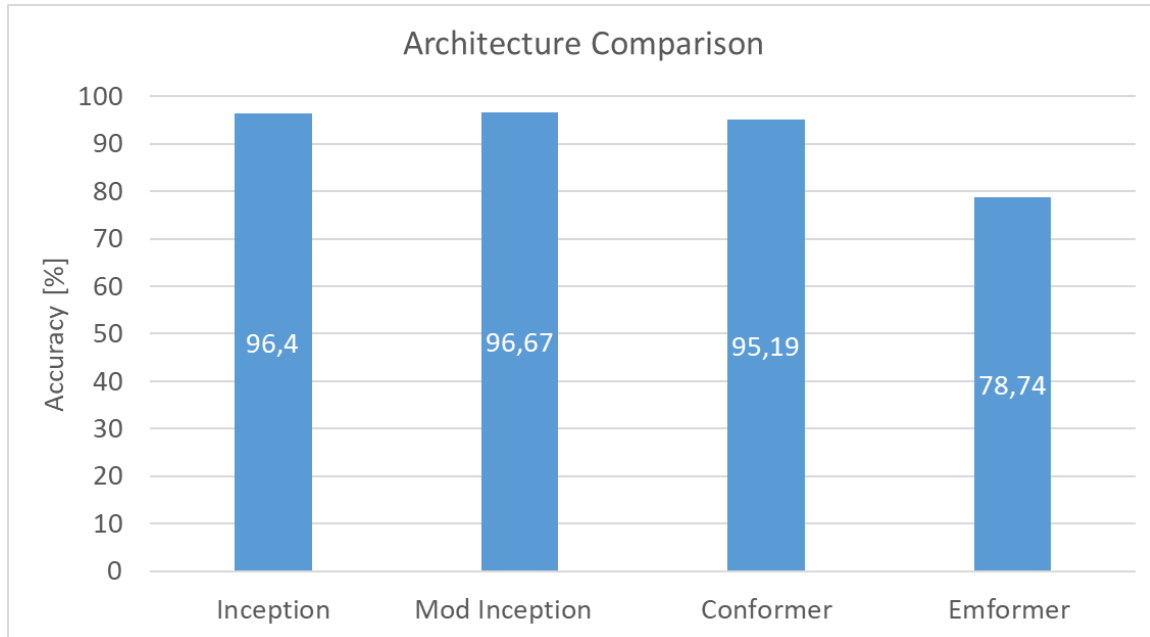


Figure 1: Accuracy results [%] of each architecture

Caution: based on the results of this experiment, the Modified Inception network was selected for all the following experiments.

4.2 Approaches tests

Figure 2 below shows the accuracy results of every tested approaches to training models:

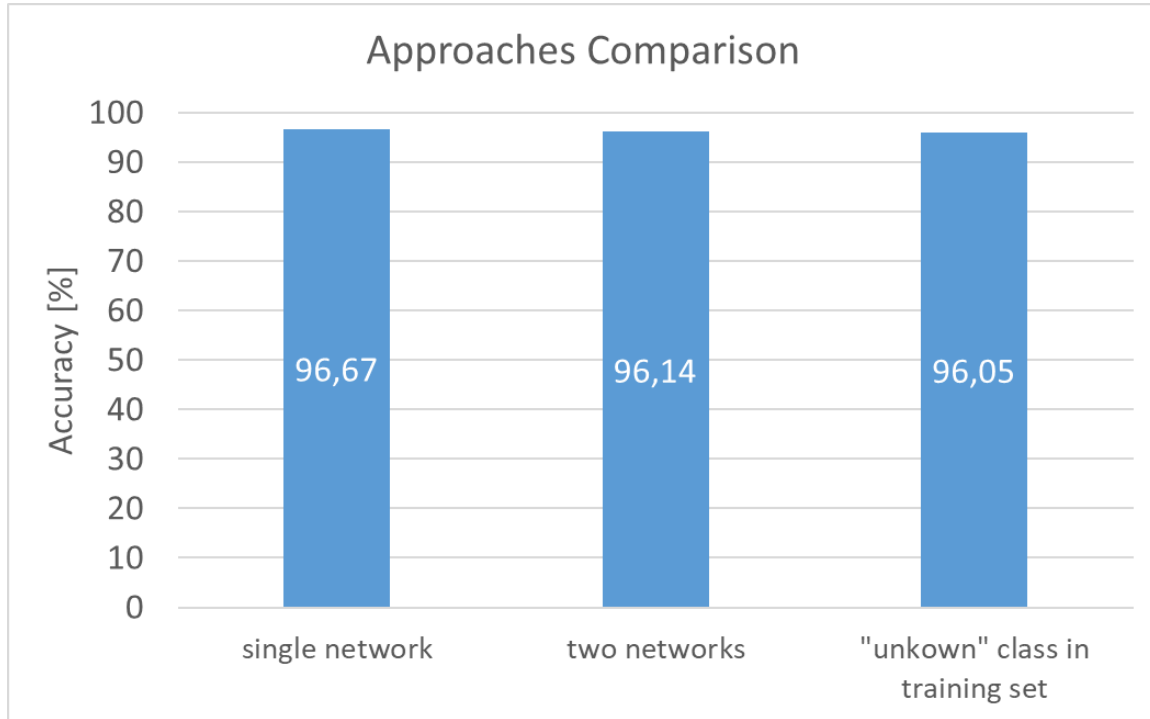


Figure 2: Accuracy results [%] of each approach

The separate network for detecting silence class had 100% accuracy, despite that the two network approach achieved lower overall results. This might be because the silence class acted as a kind of data augmentation technique and its inclusion was beneficial to the model.

The network that operated on only 12 classes also fared worse, but it might have achieved better results if there were more different "unknown" labels in the dataset.

Caution: based on the results of this experiment, for each of the following experiments we train only one model which classify all words given in train data and also silence.

4.3 Augmentation tests

In order to adjust the appropriate data augmentation, different values of the `masking_param` parameter were tested. It is assumed that the data will be processed by the `TimeMasking` and `FrequencyMasking` (SpecAug suite) functions with the same `masking_param` values for both functions. Figure 3 below shows the accuracy results of every tested `masking_param` value:

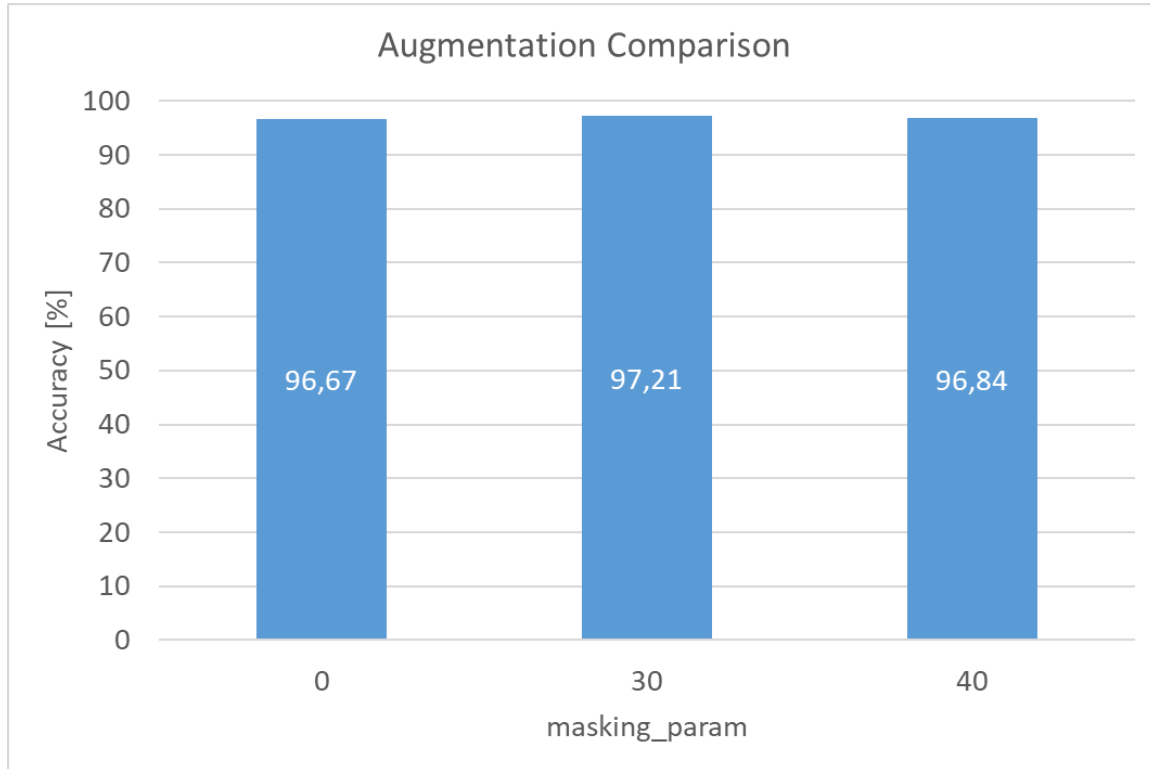


Figure 3: Accuracy results [%] of each masking_param value

Based on the results of this experiment, we set masking_param equal to 30 for the final model.

4.4 Hyperparameters tests

This section describes the hyperparameter tests performed.

4.4.1 Epoch test

We tested four different epoch limits: 20, 30, 50. These epoch limits were also equal to epochs count required for the linear scheduler to reach the lowest learning rate value. The results are presented in the Figure 4.

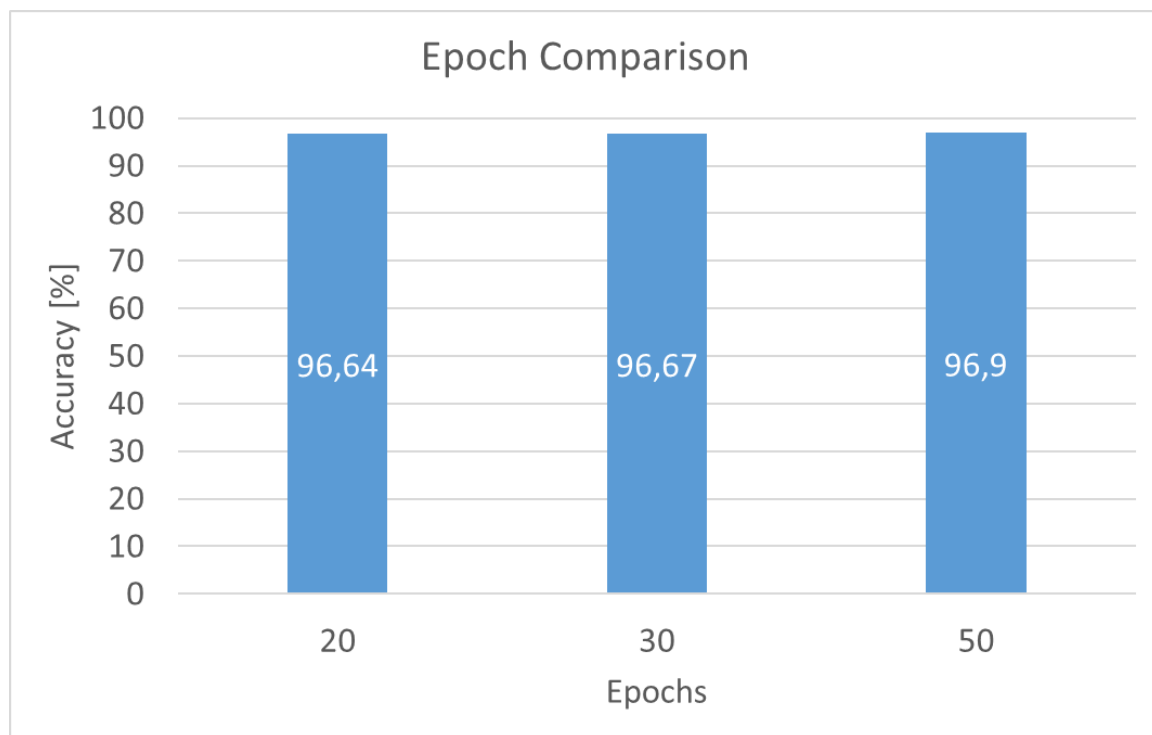


Figure 4: accuracy results [%] of each epoch limit tests

4.4.2 Batch Size test

We tested 3 batch sizes: 32, 64, 128. The results are presented in the Figure 5.

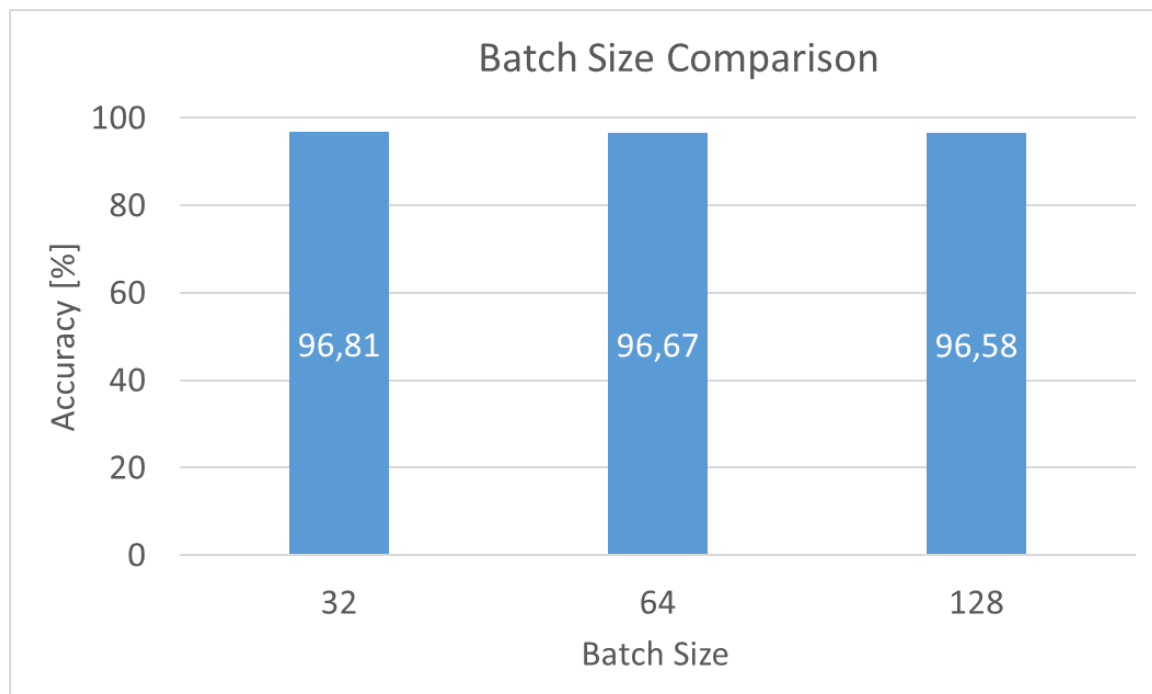


Figure 5: accuracy results [%] of each batch size

4.4.3 Learning Rate test

We tested 3 learning rate values: 0.05, 0.001, 0.005. The results are presented in the Figure 6.

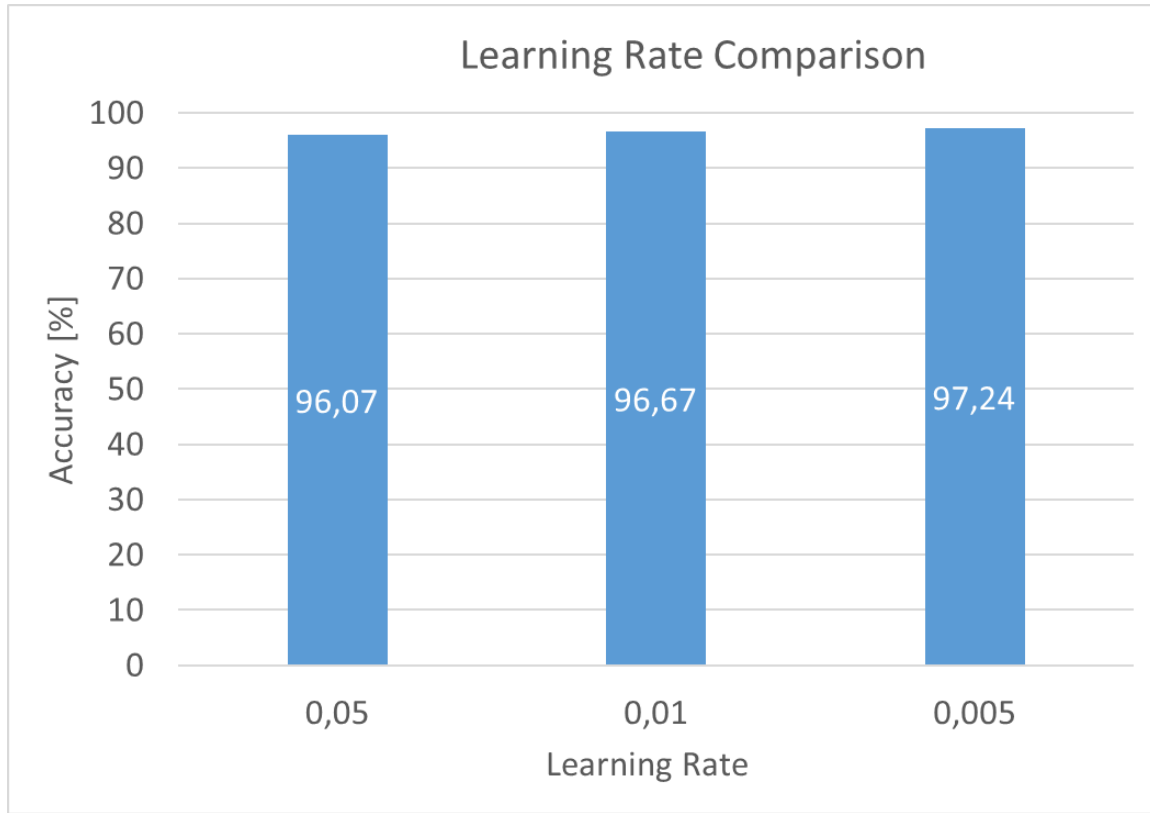


Figure 6: accuracy results [%] of each learning rate

Based on the results of this experiment, we set learning rate equal 0.005 for the final model.

4.5 Final model

Finally, the best model was tested on the test set and output was sent to kaggle. An Private Score of **88.55%**, an Public Score of **87.69%** was achieved on the test set. On the validation set, the model achieved an accuracy of 97,24%. The fact that the validation set accuracy is much higher might be because the test set was almost 25 times bigger and was more varied.

The figure 7 below shows the confusion matrix of the classification on the validation set. We can see that the the model struggled the most when differentiating word 'off' from word 'up'.

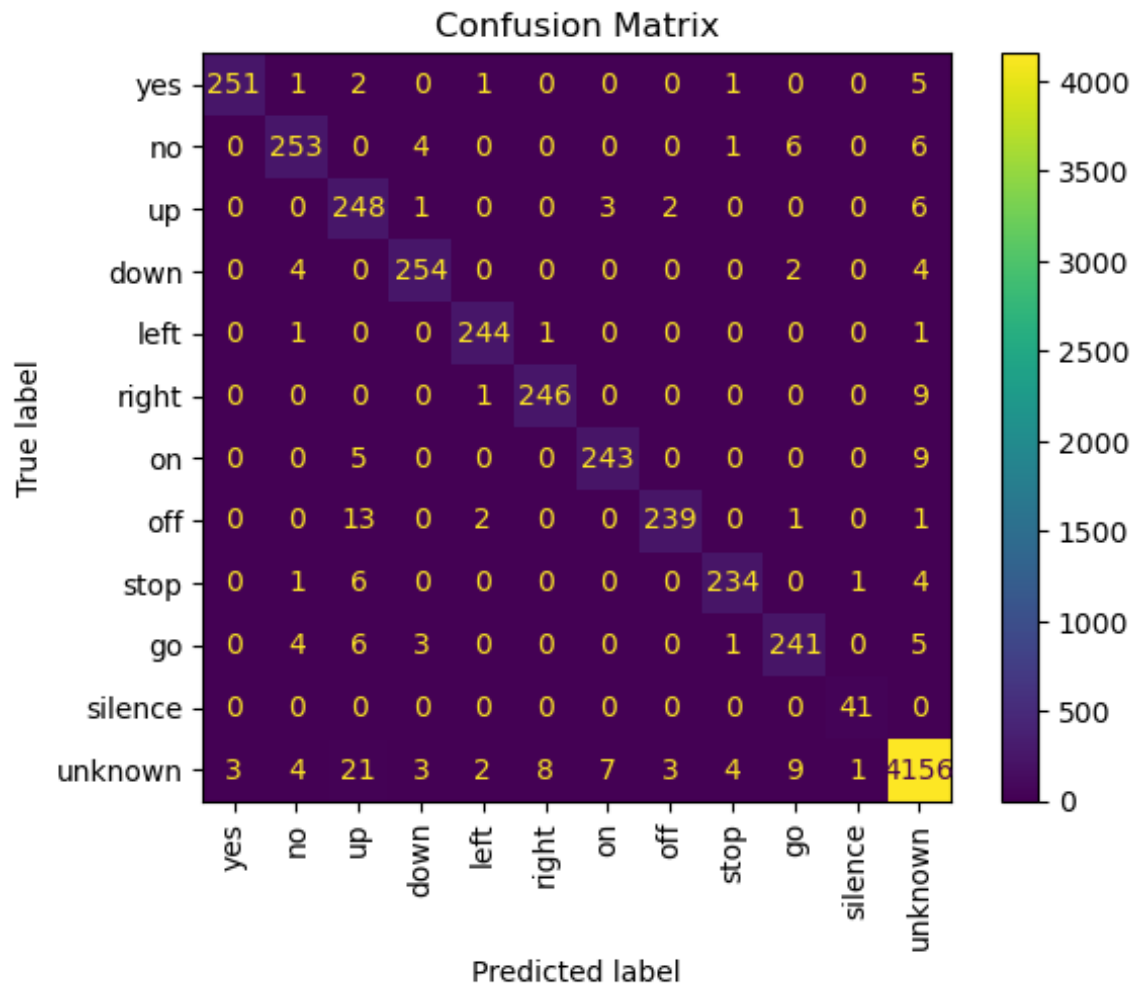


Figure 7: confusion matrix on validation set

Figure 8 shows relationship between train set loss and the number of epochs during network training.

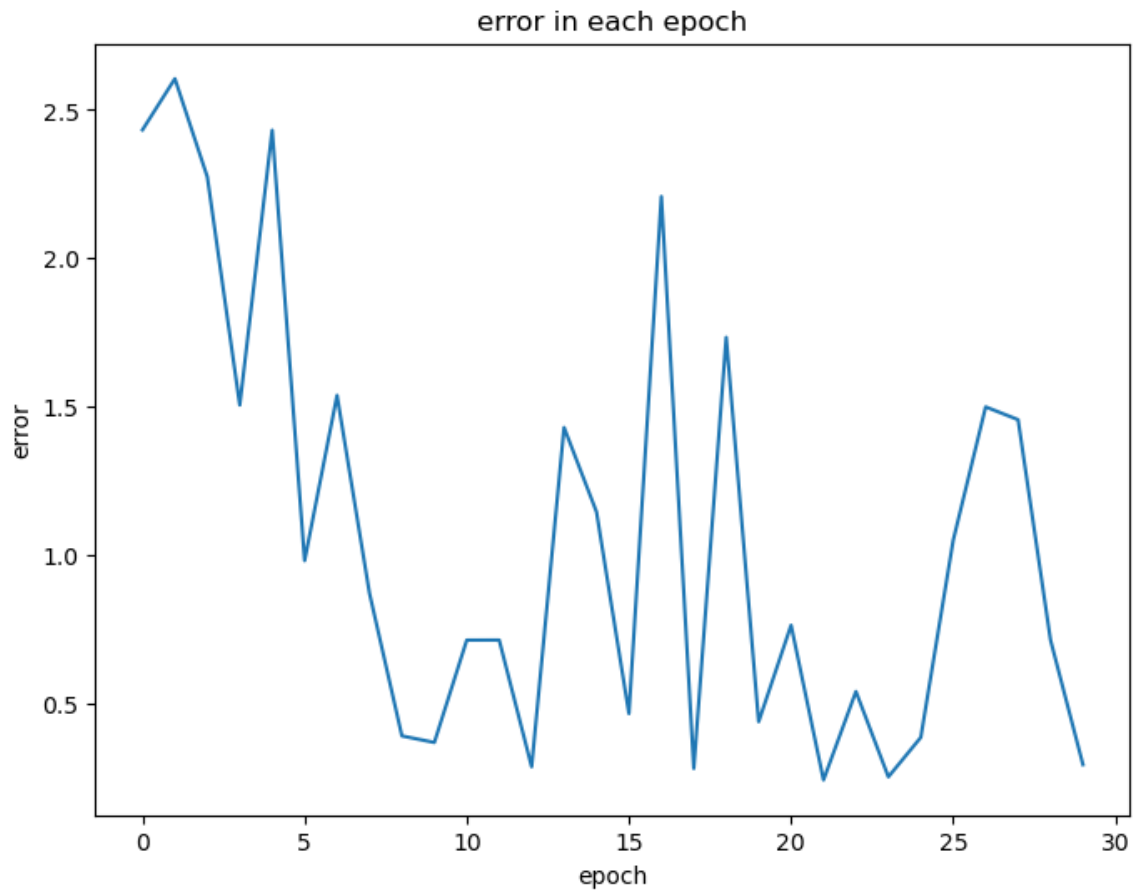


Figure 8: Relationship between train set loss and the number of epochs

Figure 9 shows relationship between model accuracy on validation set and the number of epochs during network training.

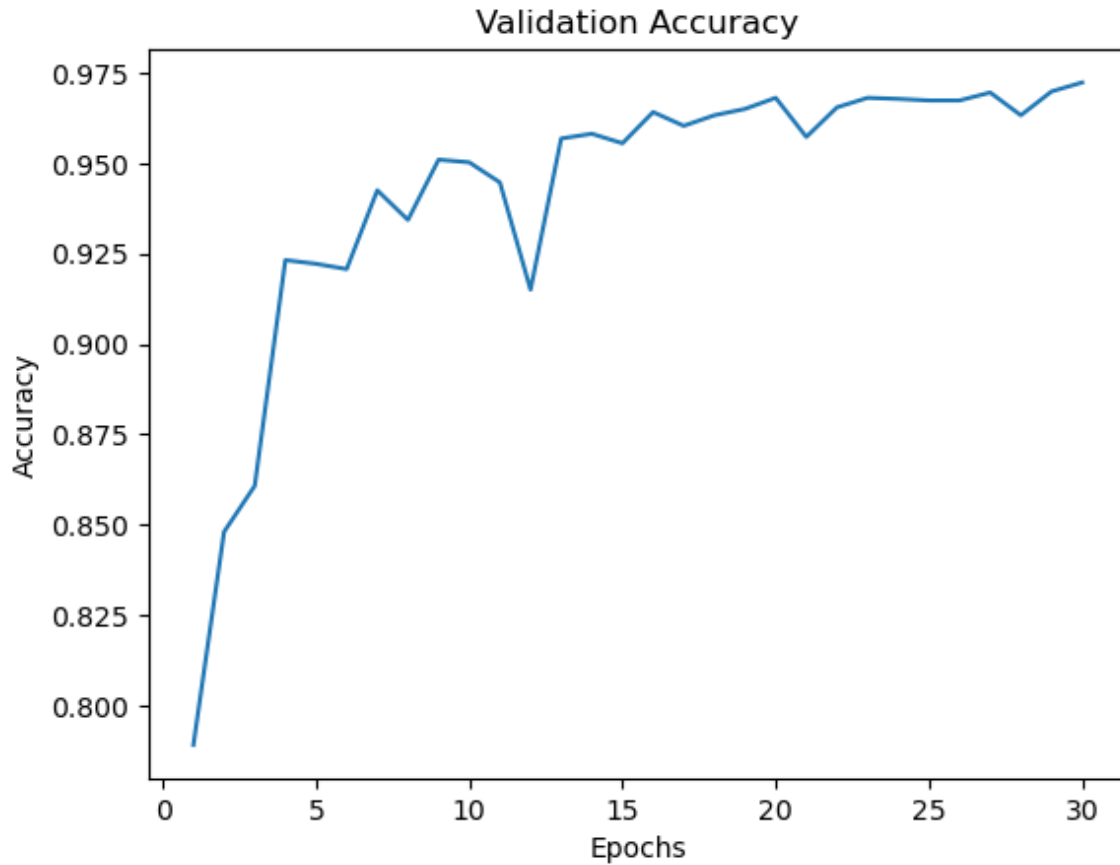


Figure 9: Relationship between validation set accuracy and the number of epochs

4.6 Ensembled model

To test the operation of ensembling, three best models were generated:

- Inception,
- Modified Inception,
- Conformer,

Subsequently, the accuracy of each model, as well as the ensemble of these three models, was compared. The comparison is presented in the Figure 10. As we can see, the ensembling process allowed to achieve better accuracy than any of the ensembled models.

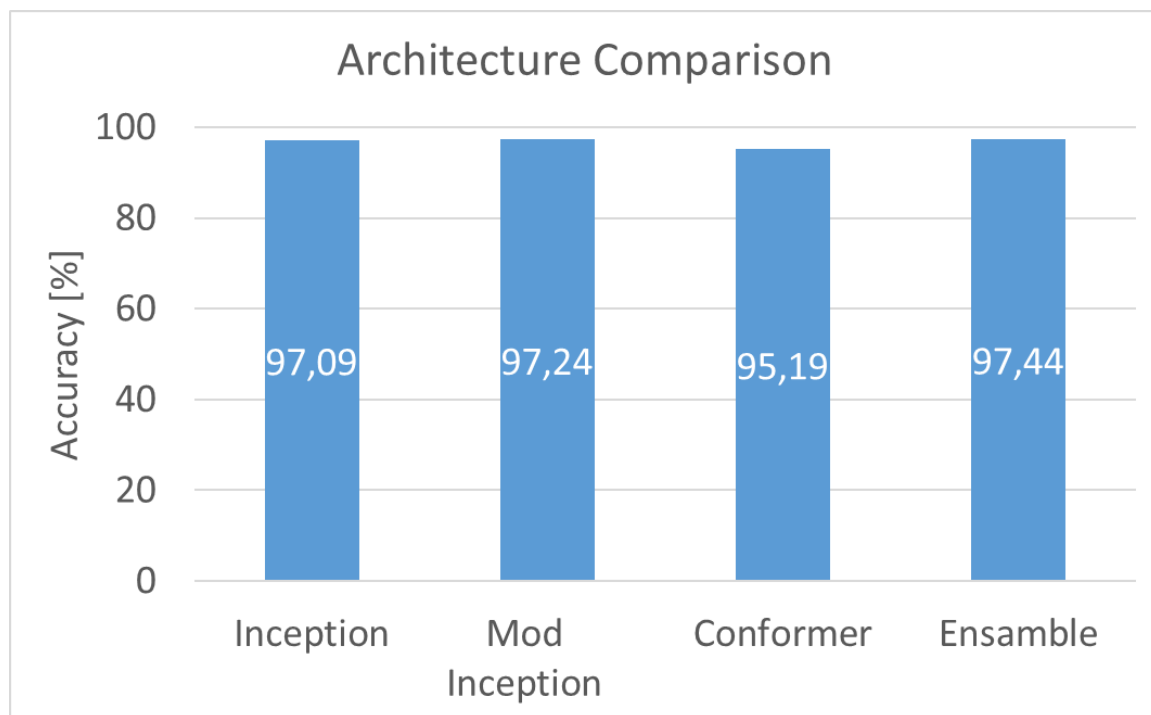


Figure 10: accuracy results [%] of architectures

This ensembled model was also sent to Kaggle and achieved Private Score of **89.06%** and Public Score of 88.19%. This is the best model that was created.

The figure 11 below shows the confusion matrix of the classification on the validation set. Once again we see that it is most difficult for models to distinguish words 'off' from word 'up'.

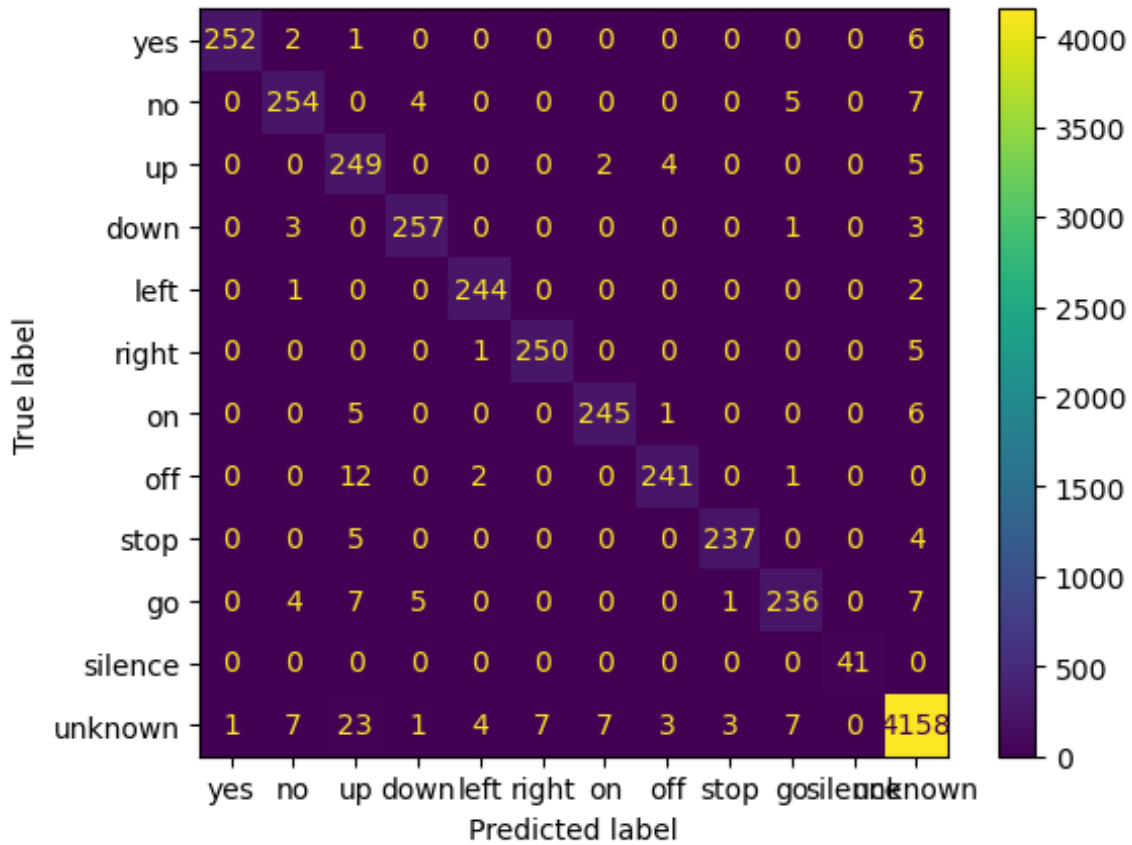


Figure 11: confusion matrix on validation set

4.7 Reproducing experiments

In order to reproduce the experiments, one needs to open the `main.ipynb` jupyter file. Then the two first sections should be run: Imports and Preparation. One should uncomment `gen_files(data_dir)` when running the code for the first time to generate the preprocessed data files. One should also change the hyperparameters in the "base hyperparameters" cell to the ones described in the report (the hyperparameters in that cell correspond to the best hyperparameters used in the final test). Afterwards, one should run the code in a section corresponding to test one wants to run (`perform_test(...)` function). The seed is already written into the code, it's defined in `experiments.py`.

5 Conclusions

As a result of the project, a high accuracy of **89.06%** on the Kaggle competition test set was achieved. On the validation set the accuracy was even higher - **97.44%**. Considering that out of over 24 thousands submissions to the Kaggle competition, the best score was 91%, which is only slightly higher than our result, the project can be considered successful.

The discrepancy between the result on the test set and the validation set might be because the test set was much bigger - it was around 3 times bigger than the training set and around 25 times bigger than the validation set. The test set probably contained harder examples and examples that weren't similar to the ones found in the training and validation sets.

Four models were tested: two transformer Emformer, Conformer, as well as InceptionTime model and our modification of it. The Conformer, InceptionTime and modified InceptionTime all achieved similar results in the range of 95-97% accuracy on the validation set, while Emformer achieved worse results at 78.74%. This could have been caused by using wrong hyperparameters' values for this model. The model out of those tested was the modified InceptionTime, which is well suited for analysis of time series and audio data. The fact it outperformed the transformer models might be because the task was fairly easy (as very high accuracy was achieved) and that all the audio files had the same length. One of the transformer's main strengths is their ability to parse data of varying lengths, which was not the case in this project.

Three approaches to classifying voice commands were considered: classifying all words and then filtering out the "unknown" labels afterwards; using a separate network for the special silence class and then classifying the rest in a second network; relabeling the labels as "unknown" before the training. Out of those, the best results were achieved by the first method. The other two methods had results worse by around 0.5 percent point. The separate network for identifying silence class had a 100% accuracy on the test set, but the second network, which didn't train on silence class, achieved worse results. This might be because the inclusion of the silence class might work as some sort of data augmentation technique and is beneficial to training the model. The network that operated on only 12 classes also fared worse, but it might have achieved better results if there were more different "unknown" labels in the dataset. This is something that could be investigated in further research, on a different dataset.

Two data augmentation techniques from SpecAug suite were tested: TimeMasking and FrequencyMasking. Inclusion of those techniques improved the accuracy of the best model by 0.5 percent point.

The influence of hyperparameter change was investigated: batch size, epoch count and learning rate. Only the learning rate changes had major influence over the results of the model. The best hyperparameters were found and used in the best model.

It is recommended for future testing to further investigate more models, preprocessing techniques and data augmentation techniques to achieve even better results. Assessing the results on a different dataset would also be recommended.

6 Bibliography

- [1] Vaswani, Ashish & Shazeer, Noam & Parmar, Niki & Uszkoreit, Jakob & Jones, Llion & Gomez, Aidan & Kaiser, Lukasz & Polosukhin, Illia. (2017). Attention Is All You Need.

- [2] inversion, Julia Elliott, Mark McDonald, Pete Warden, Raziel. (2017). TensorFlow Speech Recognition Challenge. Kaggle. <https://www.kaggle.com/c/tensorflow-speech-recognition-challenge/data>
- [3] Ismail Fawaz, Hassan & Lucas, Benjamin & Forestier, Germain & Pelletier, Charlotte & Schmidt, Daniel & Weber, Jonathan & Webb, Geoffrey & Idoumghar, Lhassane & Muller, Pierre-Alain & Petitjean, François. (2020). InceptionTime: Finding AlexNet for time series classification. *Data Mining and Knowledge Discovery*. 34. 1-27. 10.1007/s10618-020-00710-y.
- [4] InceptionTime github repository by hfawaz, <https://github.com/hfawaz/InceptionTime/blob/master/classifiers/inception.py>.
- [5] Gulati, Anmol & Qin, James & Chiu, Chung-Cheng & Parmar, Niki & Zhang, Yu & Yu, Jiahui & Han, Wei & Wang, Shibo & Zhang, Zhengdong & Wu, Yonghui & Pang, Ruoming. (2020). Conformer: Convolution-augmented Transformer for Speech Recognition. 5036-5040. 10.21437/Interspeech.2020-3015.
- [6] Shi, Yangyang & Wang, Yongqiang & Wu, Chunyang & Yeh, Ching-Feng & Chan, Julian & Zhang, Frank & Le, Duc & Seltzer, Mike. (2020). Emformer: Efficient Memory Transformer Based Acoustic Model For Low Latency Streaming Speech Recognition.
- [7] Mohamed Ahmed, Kaggle notebook, <https://www.kaggle.com/code/mohamedahmedae/inceptiontime-ctc-models-val-f1socre-0-96>.
- [8] Park, Daniel & Chan, William & Zhang, Yu & Chiu, Chung-Cheng & Zoph, Barret & Cubuk, Ekin & Le, Quoc. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition.
- [9] Oza, Urvi & Patel, Sarangi & Kumar, Pankaj. (2021). NovemE - Color Space Net for Image Classification. *Intelligent Information and Database Systems*. 531-543.